

Department of Information Engineering Technology
The Superior University, Lahore

Mobile Application Development Lab

Experiment No.12

Flutter: Adding Firebase Backend

Prepared for

By:

Name: _____

ID: _____

Section: _____

Semester: _____

Total Marks: _____

Obtained Marks: _____

Signature: _____

Date: _____

Experiment No.12

Flutter: Adding Firebase Backend

(Rubrics)

Name: _____

Roll No: _____

A. PSYCHOMOTOR

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Follow Procedures	1	0	0.25	0.5	0.75	1	
2	Software and Simulations	2	0	0.5	1	1.5	2	
3	Accuracy in Output Results	3	0	0.75	1.5	2.25	3	
Sub Total		6	Sub Total marks Obtained in Psychomotor(P)					

B. AFFECTIVE

Sr. No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Respond to Questions	1	0	0.25	0.5	0.75	1	
2	Lab Report	1	0	0.25	0.5	0.75	1	
3	Assigned Task	2	0	0.5	1	1.5	2	
Sub Total		4	Sub Total marks Obtained in Affective (A)					

Instructor Name: _____

Total Marks (P+A): _____

Instructor Signature: _____

Obtained Marks (P+A): _____

Because Google has open-sourced Flutter support for desktop and web apps, the Firebase backend services can be used for Flutter desktop and web apps, not just mobile. That's why in this chapter you'll learn how to develop a production-level mobile app.

Specifically, you'll learn how to use authentication and persist data to a cloud database by using Google's Firebase (backend server infrastructure), Firebase Authentication, and Cloud Firestore. You'll learn how to create and set up a Firebase project using Cloud Firestore as the cloud database. Cloud Firestore is a NoSQL document database to store, query, and sync data with offline support for mobile and web apps. Did I say offline? Yes, I did. The ability of a mobile app to function while an Internet connection is not available is a must-have feature that users expect. Another great feature of using Cloud Firestore is the ability to synchronize live data between devices automatically. The data syncing is fast, allowing collaboration between different devices and users. The amazing part of using these powerful features is that you don't have to deal with setting up and managing the server's infrastructure. This feature allows you to build serverless applications.

You'll configure the Firebase backend authentication provider, database, and security rules to sync data between multiple devices and platforms. To enable authentication and database services for the client-side Flutter project, you'll add the `firebase_auth` and `cloud_firestore` packages. In Chapter 15 and Chapter 16 you'll learn how to implement app-wide and local state management by using the `InheritedWidget` class and maximize platform code sharing by implementing the Business Logic Component pattern. You'll use app-wide and local state management to request data from different service classes.

WHAT ARE FIREBASE AND CLOUD FIRESTORE?

Before you start configuring, let's take a look at what Firebase encompasses. Firebase consists of one platform with a multitude of products that share and work together. Firebase handles the entire backend server infrastructure connecting iOS, Android, and web apps.

Build Apps

- **Cloud Firestore**—Store and sync NoSQL data in documents and collections between devices
- **Real-time database**—Store and sync NoSQL data as one large JSON tree between devices
- **Cloud storage**—Store and serve files
- **Cloud functions**—Run backend code
- **Authentication**—Secure user's authentication
- **Hosting**—Deliver web app assets

Ensure App Quality

- **Crashlytics**—Real-time crash reports
- **Performance monitoring**—App performance
- **Test lab**—Test apps on devices hosted by Google

Grow Business

- **In-app messaging**—Send users messages
- **Google Analytics**—Perform app analytics
- **Predictions**—User segmentation based on behavior
- **A/B testing**—Optimize app experience
- **Cloud messaging**—Send messages and notifications
- **Remote config**—Modify the app without deploying a new version
- **Dynamic links**—App deep links
- **App indexing**—Drive search traffic to a mobile app

Wow, this sounds amazing, but does it cost a fortune? Actually no; Google offers the Spark Plan, which gives you free access to the majority of the products, especially Cloud Firestore. You can view the details and restrictions at <https://firebase.google.com/pricing>.

In this chapter, I'll focus on using Cloud Firestore to store and sync data between devices. For general information on Firebase, you can browse <https://firebase.google.com>.

What is Cloud Firestore? It stores data in documents organized in collections, similar to JSON. It can scale complex and hierarchical data by using subcollections within documents. You'll take a detailed look at it in the next section, "Structuring and Data Modeling Cloud Firestore." It offers offline support for iOS, Android, and web apps. For the iOS and Android platforms, the offline persistence is enabled by default, but for the Web, the offline is disabled by default. To optimize querying data, it supports indexed queries with sorting and filtering. It can use transactions that automatically repeat until the task is completed. Data scaling is automatically handled for you. You use mobile SDKs to integrate different features of Firebase products into your apps.

Structuring and Data Modeling Cloud Firestore

To understand the Cloud Firestore data structure, let's compare it to a standard SQL Server database (see Table 14.1). The SQL Server database is a relational database management system (RDBMS) that supports table and row data modeling (Figure 14.1). The comparison is not a one-to-one but a guideline since the data structures differ.

TABLE 14.1: Data structure comparison

SQL SERVER DATABASE	CLOUD FIRESTORE
Table	Collection
Row	Document
Columns	Data

SQL Database

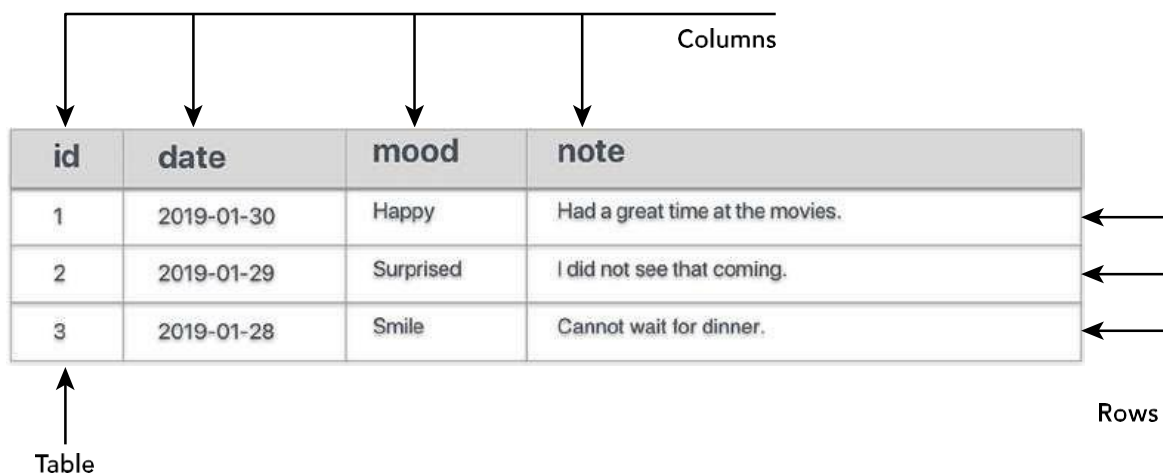


FIGURE 14.1: SQL Server Database Data Model

In Cloud Firestore, a collection can contain only documents. A document is a key-value pair and can optionally point to subcollections. Documents cannot point to another document and must be stored in collections (Figure 14.2).

What is the collection's responsibility? Collections are containers for documents; they hold them the same way a folder holds pages.

What is the document's responsibility? Documents hold data that is stored as a key-value pair similar to JSON. Documents support extra data types that JSON does not support. Each document is identified by name, and they are limited to 1MB in size (see Table 14.2).

Cloud Firestore

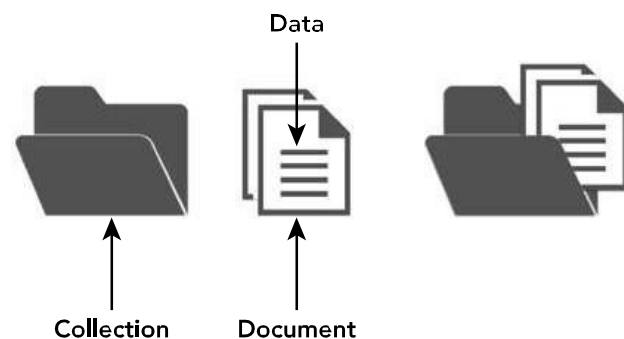


FIGURE 14.2: Cloud Firestore Data Model

TABLE 14.2: Cloud Firestore sample data

TYPE	VALUE
Collection	journals
Document	R5NcTWAaWtHTttYtPoOd
Document data as key-value pair	date: "2019-0202T13:41:12.537285" mood: "Happy" note: "Great movie." uid: "F1GGeKiwp3jRpoCVskdBNmO4GUN4"

Let's take a look at the Cloud Firestore sample data as JSON objects and in the Cloud Firestore console (Figure 14.3). Notice that the document name is a unique ID that can be automatically created by Cloud Firestore, or you can manually generate it.

```
{
  "journals": [
    {
      "R5NcTWAaWtHTttYtPoOd1": {
        "date": "2019-0202T13:41:12.537285",
        "mood": "Happy",
        "note": "Great movie.",
        "uid": "F1GGeKiwp3jRpoCVskdBNmO4GUN4"
      }
    }
  ]
}
```

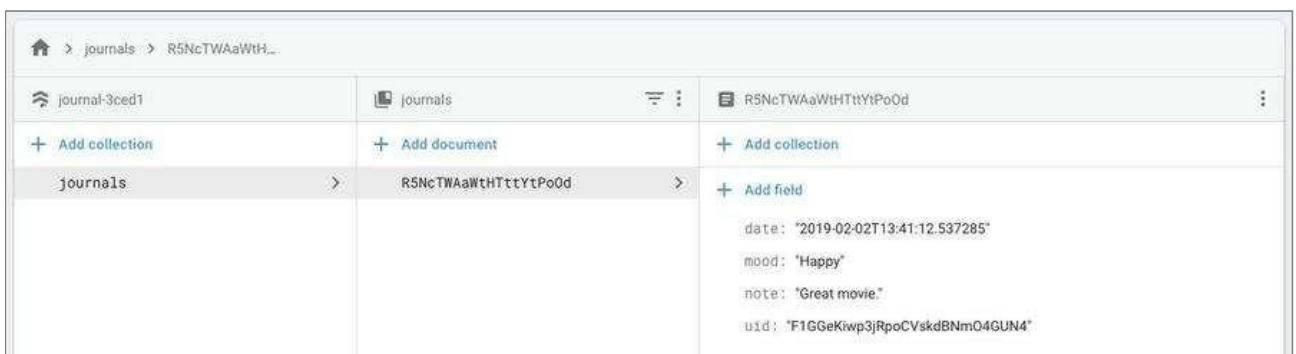


FIGURE 14.3: Collection and Document

Cloud Firestore supports many data types such as array, Boolean, byte, date and time, floating-point number, geographical point, integer, map, reference, text string, and null. One of the major advantages of using Cloud Firestore is the automatic syncing of data between devices and the ability of the client app to continue working offline while the Internet is not available.

Viewing Firebase Authentication Capabilities

Adding security to an app is extremely important in keeping the information private and safe. Firebase Authentication provides built-in backend services accessible from the client's SDK to support full authentication. The following is a list of the currently available authentication sign-in providers:

- Email/Password
- Phone
- Google
- Play Games (Google)
- Game Center (Apple)
- Facebook
- Twitter
- GitHub
- Anonymous

Each authentication sign-in provider is disabled by default, and you enable the providers needed for your app specs (Figure 14.4). I'll walk you through how to enable a sign-in provider later in this chapter.

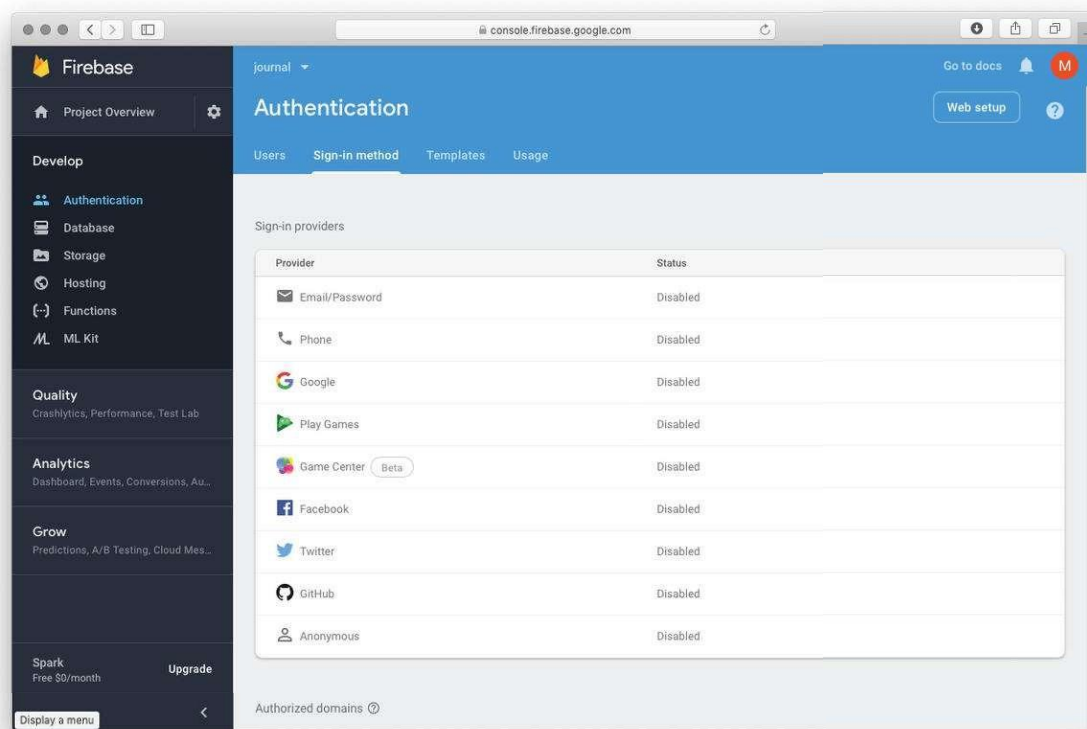


FIGURE 14.4: Firebase Authentication's Sign-In Providers

An important note on using the Anonymous provider is that if the user deletes the app from the device and reinstalls it, a new anonymous user is created and will not have access to the previous data. The data from the original anonymous user is still stored on the backend, but the client app has no way of knowing the original anonymous user ID since the app was deleted from the device. One example of using the Anonymous sign-in provider is to allow the user free access to the app and allow a paid upgrade path to advanced features.

From your mobile app, you retrieve the user's authentication credentials from any of the available sign-in providers. You pass these credentials to the client Firebase Authentication SDK, and the Firebase backend services verify whether credentials are valid and return a response to the client app. If the credentials are valid, you allow the user to access the data and pages in the app depending on the security rules that you will learn about in the next section, "Viewing Cloud Firestore Security Rules."

Once a particular sign-in provider is enabled, you use the Firebase Authentication SDK to create a Firebase User object saved on the Firebase backend. The Firebase User object has a set of properties that you can customize, except for the unique ID. You can customize the primary email address, name, and a photo URL (usually the user's photo or an avatar). By using the Firebase User object's unique ID, you bind all the data that the user has created to the ID.

Viewing Cloud Firestore Security Rules

To secure access to collections and documents, you implement Cloud Firestore security rules. In the previous section, you learned that you create a Firebase User object through the Firebase Authentication SDK. Once you have the Firebase User object's unique ID, you use it with Cloud Firestore security rules to secure and lock data to each user. The following code shows the security rules that you'll create for securing the Cloud Firestore database:

```
service cloud.firestore {
  match /databases/{database}/documents {
    match /journals/{document=**} {
      allow read, write: if resource.data.uid == request.auth.uid;
      allow create: if request.auth.uid != null;
    }
  }
}
```

The rules are editable from the database rules page on the Firebase console website. The rules consist of using the `match` statements to identify documents, with the `allow` expressions to control access to the documents. Every time you change the rules and save them, a history of changes is automatically created, allowing you to revert changes if needed (Figure 14.5).

Let's take a look at an example that requires rules to allow a user to read and write documents assigned to them. The first `match /databases/{database}/documents` declaration tells the rules to match any Cloud Firestore Database in the project.

```
match /databases/{database}/documents
```

The second and the main part of understanding is to use the `match` statement to point to the collection and expression to evaluate, as in `match /journals/{document=**}`. The `journals` declaration is the container name, and the expression to evaluate is `document=**` (all documents for the

journals collection) wrapped inside curly brackets. Inside this particular match, you use the `allow` expression for the `read` and `write` privileges by using the `if` statement to see whether the `resource.data.uid` value equals the `request.auth.uid`. The `resource.data.uid` is the `uid` field inside the document, and the `request.auth.uid` is the logged-in user's unique ID.

```
match /journals/{document=**} {  
  allow read, write: if resource.data.uid == request.auth.uid;  
}
```

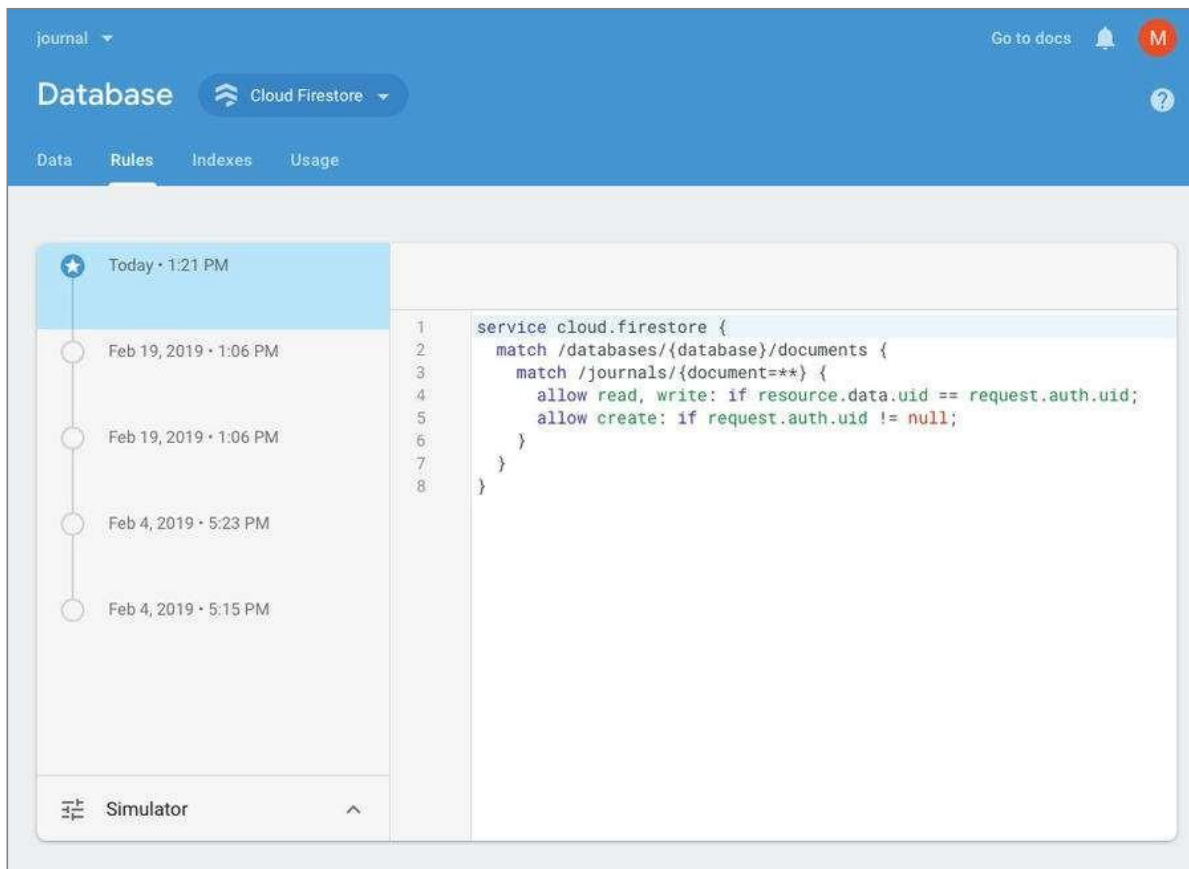


FIGURE 14.5: Cloud Firestore Rules

You can break down the rules even further by using a `match` statement for each `read` or `write` action. For the `read` rule, it can be broken into `get` and `list`. For the `write` rule, it can be broken into `create`, `update`, and `delete`.

```
//allow Readget: if <condition>;  
  allow list: if <condition>;  
  
\\Writecreate: if <condition>;  
  allow update: if <condition>;  
  allow delete: if <condition>;
```

The following example uses the `create` rule to allow only authenticated users to add new records by checking whether the `request.auth.uid` value is not equal to a `null` value:

```
allow create: if request.auth.uid != null;
```

CONFIGURING THE FIREBASE PROJECT

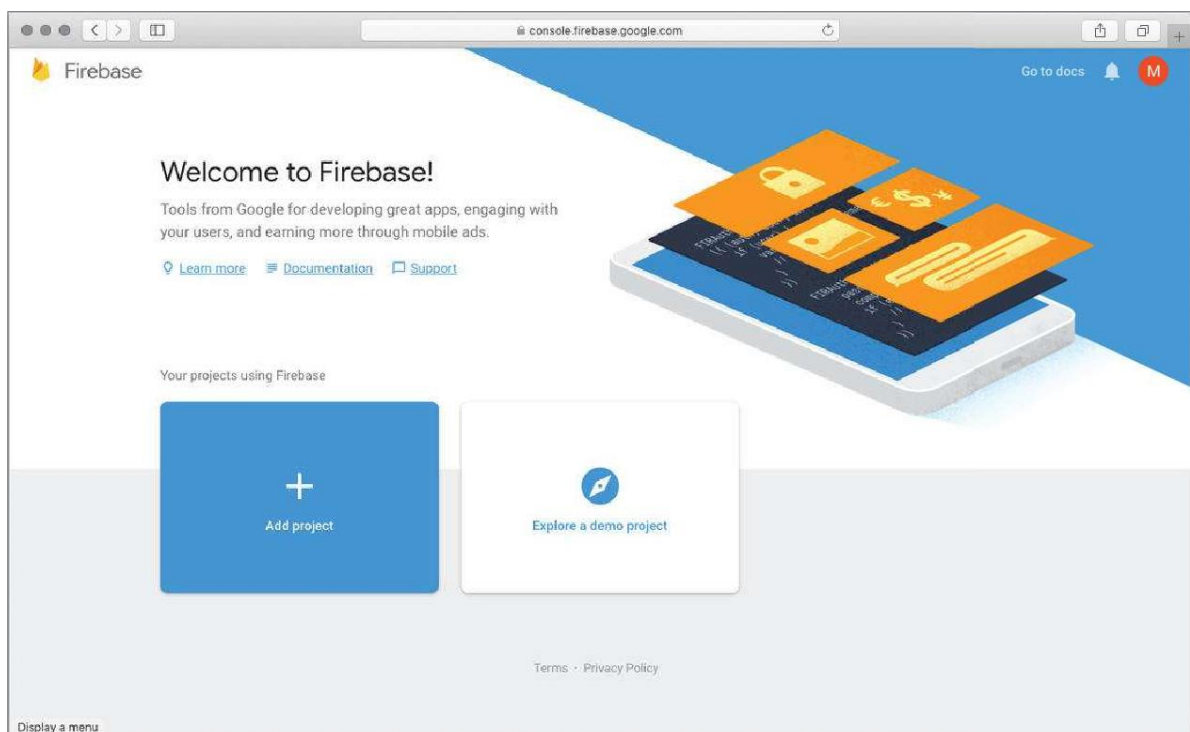
You now understand how Cloud Firestore stores data and the benefits of syncing multiple devices with offline data persistence. The offline feature works by caching a copy of the app's active data, making it accessible when the device is offline. Before you can use Cloud Firestore in your app, you need to create a Firebase project.

A Firebase project is backed by the Google Cloud Platform, which allows apps to scale. The Firebase project is a container that supports sharing features such as the database, notifications, users, remote config, crash reports, and analytics (many more) between the iOS, Android, and web apps. Each account can have multiple projects, for example to separate different and unrelated applications.

TRY IT OUT Creating the Firebase Project

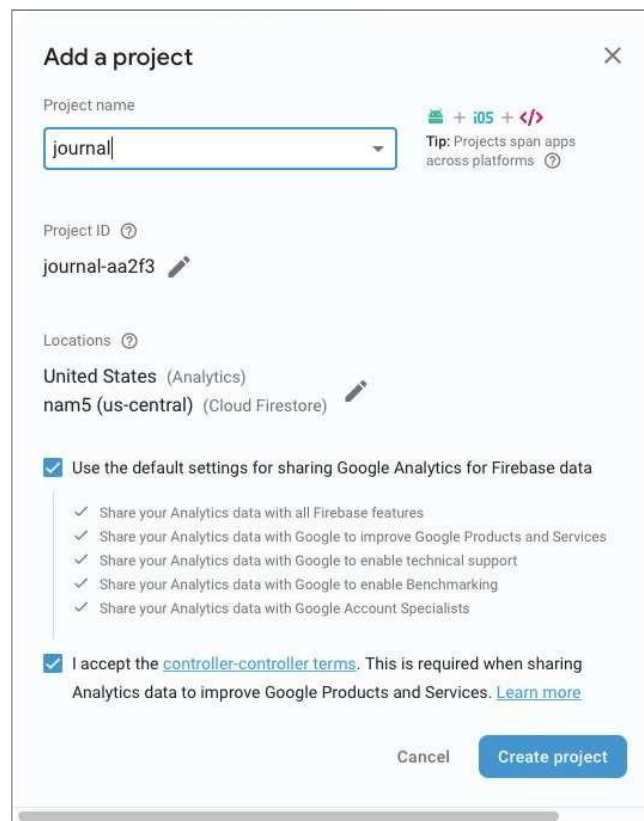
You need to create a Firebase project that sets up a container to start adding your Cloud Firestore database and enabling authentication. You will start by adding the iOS app, and then you'll continue by adding the Android app.

1. Navigate to <https://console.firebase.google.com> and log in to Google Firebase with your Google account. If you do not have a Google account, you can create one at <https://accounts.google.com/SignUp>.



2. Click the Add Project button in Firebase; the Add A Project dialog will open. For the project name, enter journal; the project ID is automatically created for you. Notice it takes the project name and adds a unique identifier to it like journal-aa2f3. (Your ID will be different because each project name must be unique.)

The location of the Cloud Firestore project is automatically selected for you, but you can change it if you like. You will need to select each checkbox to accept the Google Analytics terms; then click the Create Project button.



Add a project [X]

Project name
journal

+ iOS + </>
Tip: Projects span apps across platforms. ?

Project ID ?
journal-aa2f3

Locations ?
United States (Analytics)
nam5 (us-central) (Cloud Firestore)

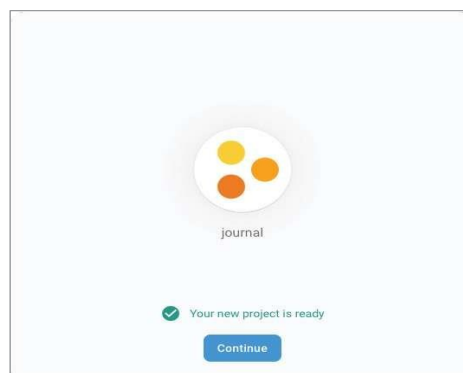
☒ Use the default settings for sharing Google Analytics for Firebase data

- ✓ Share your Analytics data with all Firebase features
- ✓ Share your Analytics data with Google to improve Google Products and Services
- ✓ Share your Analytics data with Google to enable technical support
- ✓ Share your Analytics data with Google to enable Benchmarking
- ✓ Share your Analytics data with Google Account Specialists

☒ I accept the [controller-controller terms](#). This is required when sharing Analytics data to improve Google Products and Services. [Learn more](#)

Cancel Create project

3. When you are presented with the dialog showing that the new project is ready, click the Continue button.

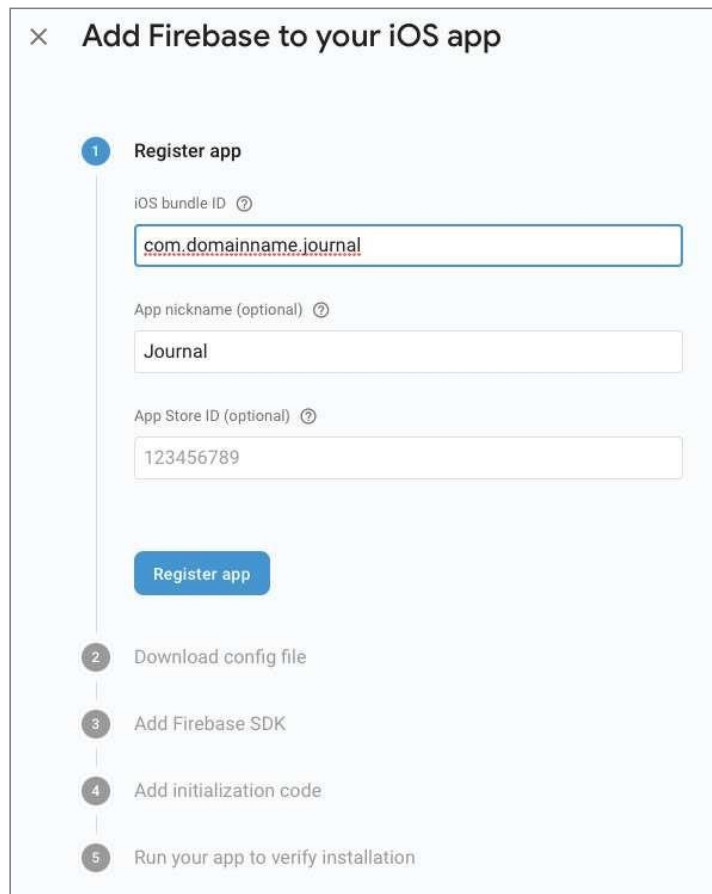


iOS

4. From the main Firebase project page, click the iOS button to add Firebase to the iOS app that you created in the section “Adding Authentication and Cloud Firestore Packages to the Client App.”

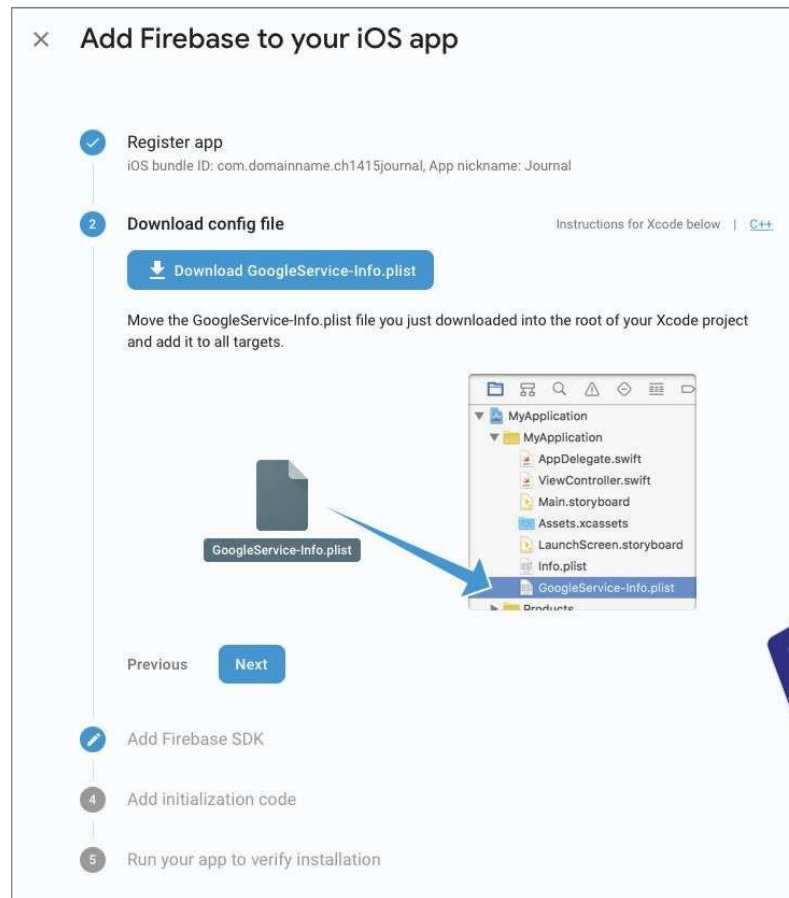


5. Enter the iOS bundle ID, as in `com.domainname.journal`. The bundle ID is the reverse domain name, *com.domainname*, combined with the Flutter app name of *journal*.
6. Enter the optional app nickname *Journal* and skip the optional App Store ID since this is obtained once an app is created from Apple's iTunes Connect to submit an app for distribution approval. Click the Register App button.

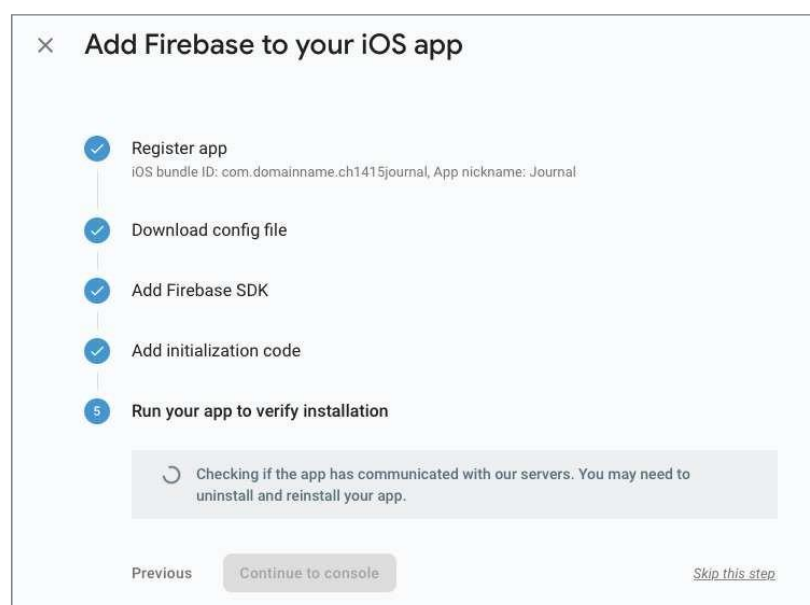


The screenshot shows a dialog box titled "Add Firebase to your iOS app" with a close button (X) in the top left corner. A vertical progress bar on the left indicates five steps: 1. Register app (active), 2. Download config file, 3. Add Firebase SDK, 4. Add initialization code, and 5. Run your app to verify installation. The "Register app" section contains three input fields: "iOS bundle ID" with the value `com.domainname.journal`, "App nickname (optional)" with the value `Journal`, and "App Store ID (optional)" with the value `123456789`. A blue "Register app" button is located below the input fields.

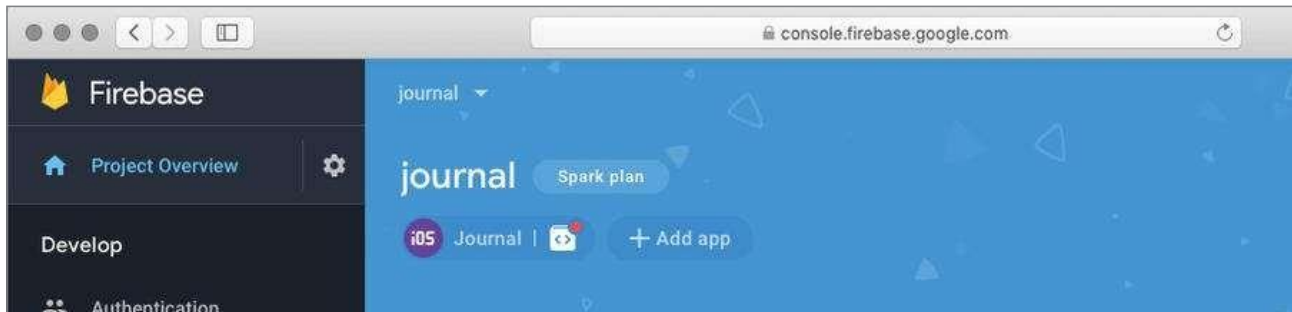
7. Click the Download `GoogleService-Info.plist` button. (You'll add the downloaded `GoogleService-Info.plist` file to your Xcode project in the section "Adding Authentication and Cloud Firestore Packages to the Client App.")
8. Click the Next button, and skip the Add Firebase SDK and Add Initialization Code steps. You are skipping these steps because the Firebase SDK is added in the section "Adding Authentication and Cloud Firestore Packages to the Client App."



9. Click Skip This Step in the Run Your App To Verify Installation step.



10. On the main Firebase project page, click the Add App button.



ANDROID

11. Click the Android button to add Firebase to the Android project that you created in the section “Adding Authentication and Cloud Firestore Packages to the Client App.”



12. Enter the Android package name, such as `com.domainname.journal`. The bundle ID is the reverse domain name, `com.domainname`, combined with a Flutter app name of `journal`. You'll create the Flutter app in the section “Adding Authentication and Cloud Firestore Packages to the Client App.”
13. Enter the optional app nickname Journal and skip the step of optional debug signing certificate SHA-1. Click the Register App button.

× Add Firebase to your Android app

1 Register app

Android package name ⓘ

App nickname (optional) ⓘ

Debug signing certificate SHA-1 (optional) ⓘ

Required for Dynamic Links, Invites, and Google Sign-In or phone number support in Auth. Edit SHA-1s in Settings.

Register app

2 Download config file

3 Add Firebase SDK

4 Run your app to verify installation

14. Click the Download googleservices.json button. (You'll add the downloaded googleservices.json file to your Android project in the section "Adding Authentication and Cloud Firestore Packages to the Client App.")
15. Click the Next button and skip the Add Firebase SDK step. You are skipping this step because the Firebase SDK is added in the section "Adding Authentication and Cloud Firestore Packages to the Client App."

×

Add Firebase to your Android app

✓

Register app

Android package name: com.domainname.ch1415journal, App nickname: Journal

2

Download config file

Instructions for Android Studio below | [C++](#)

Download google-services.json

Switch to the **Project** view in Android Studio to see your project root directory.

Move the google-services.json file you just downloaded into your Android app module root directory.



google-services.json

Previous

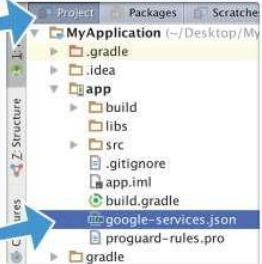
Next

3

Add Firebase SDK

4

Run your app to verify installation



16. Click Skip This Step in the Run Your App To Verify Installation step.

×

Add Firebase to your Android app

✓

Register app

Android package name: com.domainname.ch1415journal, App nickname: Journal

✓

Download config file

✓

Add Firebase SDK

4

Run your app to verify installation

✓

Checking if the app has communicated with our servers. You may need to uninstall and reinstall your app.

Previous

Continue to console

[Skip this step](#)

You will be automatically brought to the main Firebase project page, showing the iOS and Android projects you just added.



HOW IT WORKS

Navigate to the Firebase console panel at <https://console.firebase.google.com> to add or select existing projects. When you add a project and give it a name, a unique project ID is automatically created with the option to rename it. Once the project is created, the project ID cannot be changed. The server's location is automatically chosen, but you have the option to change it.

Once a Firebase project is created, you register the iOS and Android projects to use Firebase. To register each project, you enter **com.domainname.journal** for the iOS bundle ID and the Android package name. For the iOS project, you download the `GoogleService-Info.plist` file and for the Android project the `googleservices.json` file. These files are added to the Flutter app in the Xcode and Android projects in the section “Adding Authentication and Cloud Firestore Packages to the Client App.”

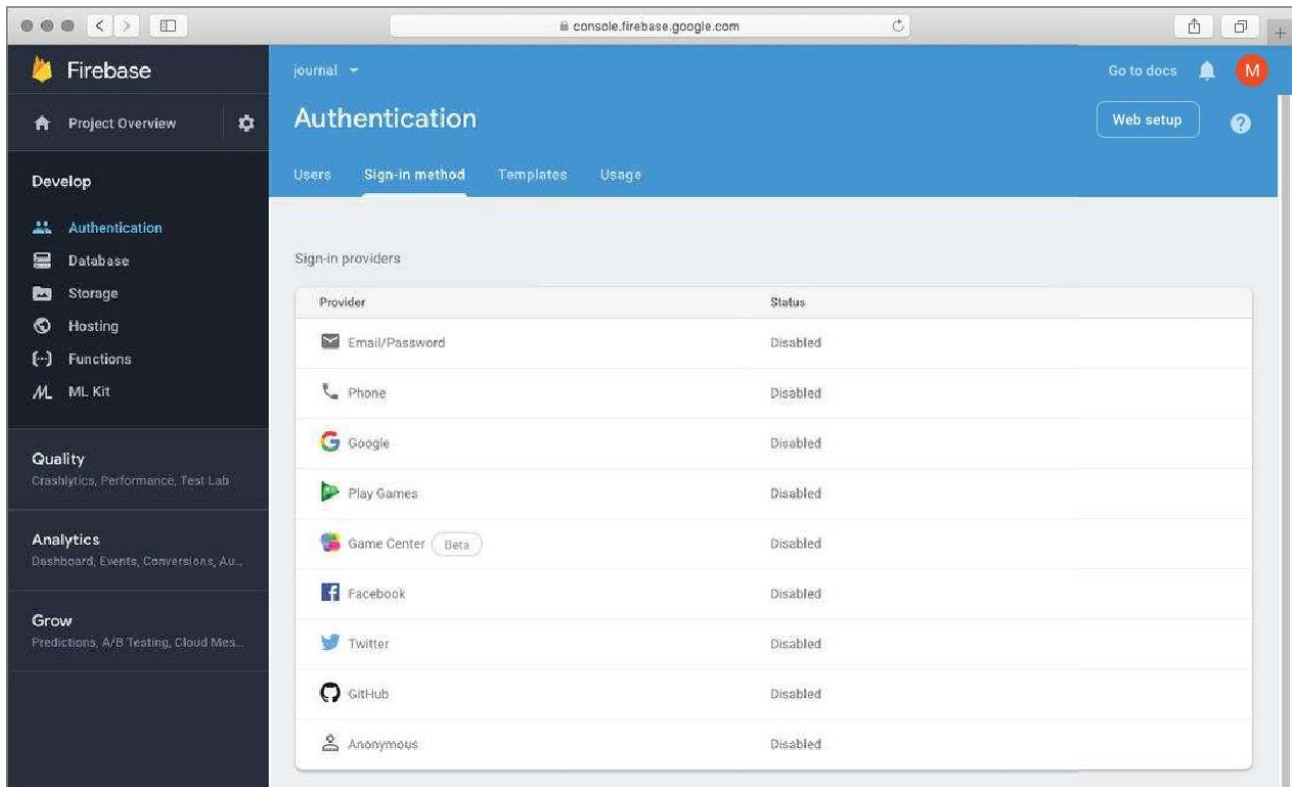
ADDING A CLOUD FIRESTORE DATABASE AND IMPLEMENTING SECURITY

You’ve learned how to create a Firebase project, making it possible to add the Cloud Firestore database and Firebase Authentication. You learned about the different available sign-in methods, and in this section, you’ll walk through how to implement security rules and enable an authentication sign-in method—specifically, an email/password authentication provider.

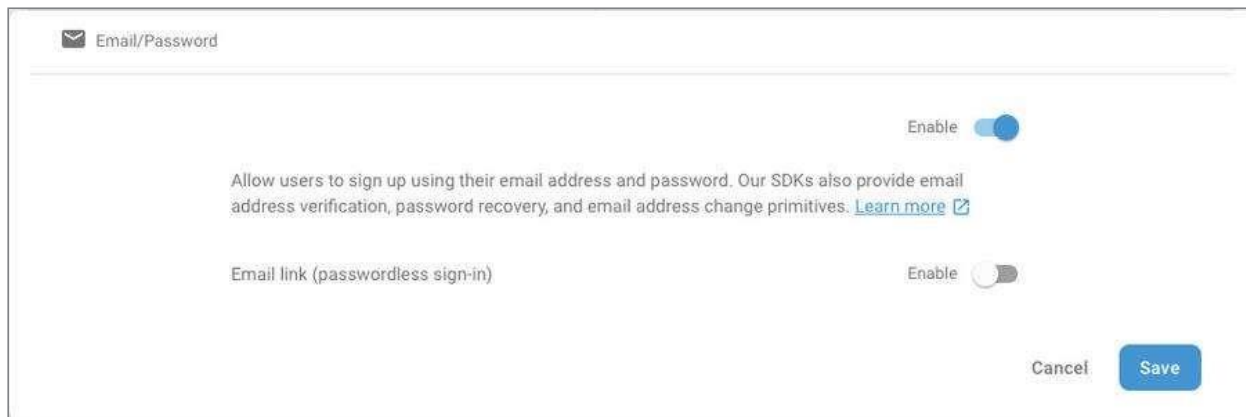
TRY IT OUT Creating the Cloud Firestore Database and Enabling Authentication

In this exercise, you’ll learn how to enable Firebase Authentication, create a Cloud Firestore database, and implement security rules to keep the data private for each user.

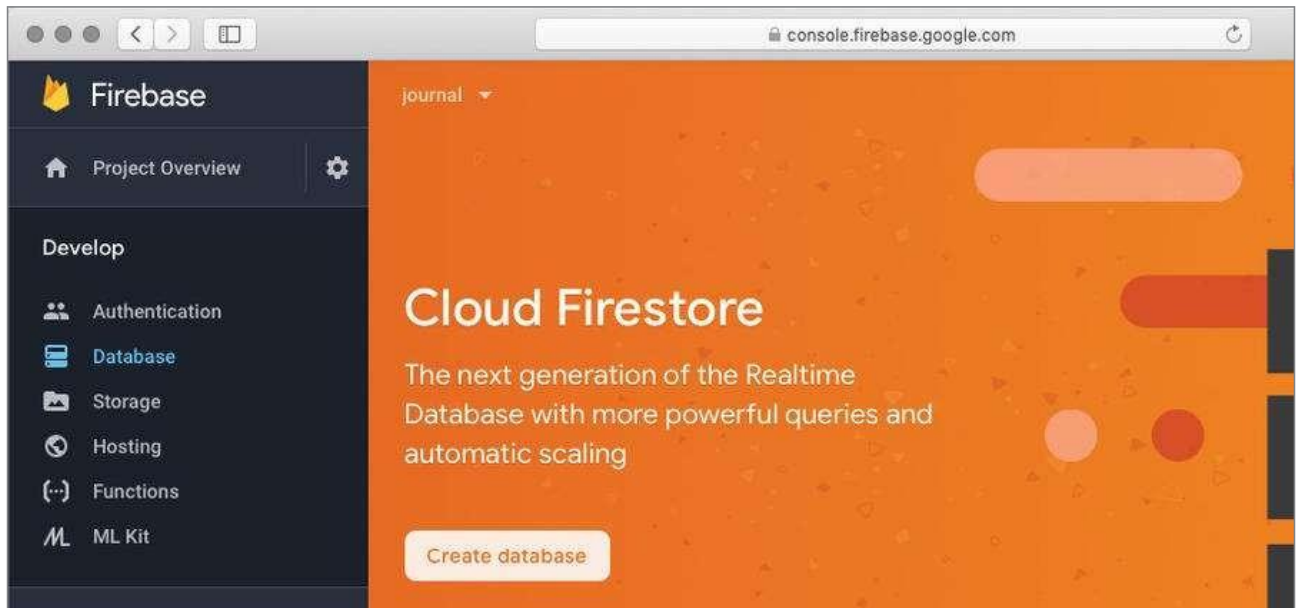
1. Navigate to <https://console.firebase.google.com> and select the journal project.
2. From the menu on the left, click the Authentication link in the Develop section. If the Develop section is closed, click the Develop link to open the submenu. Click the Sign-in Method tab showing a list of available sign-in providers.



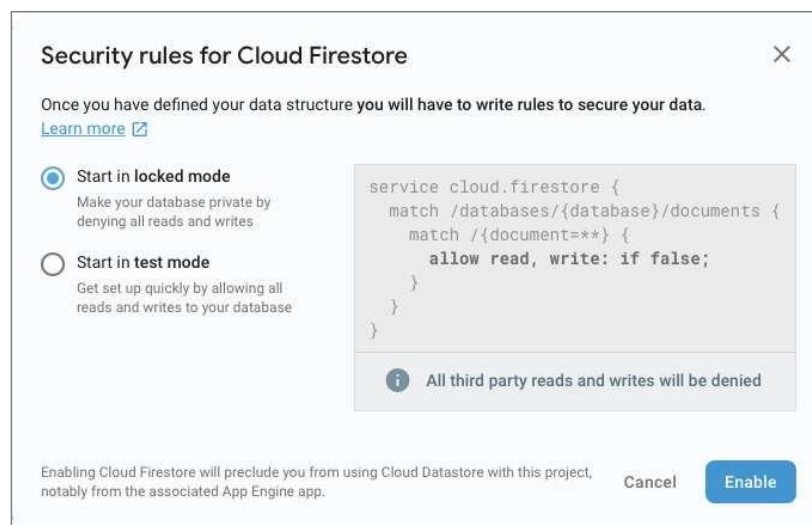
3. Click the Email/Password option, click Enable to turn on the feature, and click the Save button.



4. In the left menu, click the Database link and click the Create Database (Cloud Firestore) button.



5. In the Security Rules For Cloud Firestore dialog, leave the locked mode radio button selected and click the Enable button. The locked mode creates the basic security rules with the access locked for reading and writing privileges.



The following code shows the default security rules that are automatically created when you click the Enable button. The current security rules deny all read and write requests, making it completely safe but also inaccessible, which you will modify in step 6.

```

service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}

```

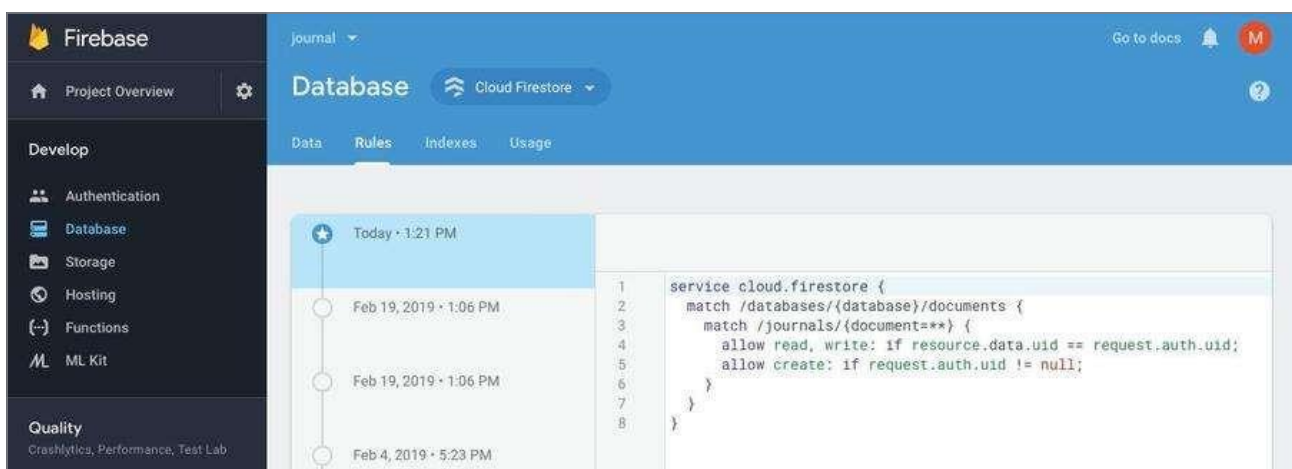
6. Tap the Rules tab to edit the default locked rules and change `match /{document=**}` to `match /journals/{document=**}`. `{document=**}` matches any document, but you are going to use a granular approach by matching the `journals` collection and `document` to the logged-in user ID. This approach allows you to restrict each document to each user ID, keeping the data secure for the rightful owner of the data.
7. Change `allow read, write: if false;` to `allow read, write: if resource.data.uid == request.auth.uid;`, restricting the data field `uid` to match the logged-in `uid`. The `resource.data.uid` is the `uid` field inside the document, and the `request.auth.uid` is the logged-in user unique ID. Add `allow create: if request.auth.uid != null;` to allow creating new records if the user is authenticated.

```

service cloud.firestore {
  match /databases/{database}/documents {
    match /journals/{document=**} {
      allow read, write: if resource.data.uid == request.auth.uid;
      allow create: if request.auth.uid != null;
    }
  }
}

```

Note that a history log of changes is automatically created, allowing the reverting of changes if needed.



Solution:

Your project

Project name

tdsip

Project ID

tdsip-c4128

Project number

307691944015

Environment

This setting customizes your project for different stages of the app lifecycle

Environment type

Unspecified

Your apps

Android apps

my_app

com.example.my_app

Apple apps

my_app (ios)

com.example.myApp

Web apps

my_app (web)

Web App

my_app (windows)

Web App

SDK setup and configuration

Need to reconfigure the Firebase SDKs for your app? Revisit the SDK setup instructions or just download the configuration file containing keys and identifiers for your app.

See SDK instructions

google-services.json

App ID

1:307691944015:android:a06bc2716eb3a90679923a

App nickname

my_app

Package name

com.example.my_app

SHA certificate fingerprints

Type

Add fingerprint

Remove this app

🏠 > chats > clkUXuR22pZtu...		
🏠 (default)	📁 chats	📄 clkUXuR22pZtusVMOYUTiir3WsM2_hOr64EyujeZJNN067cN... ⋮
+ Start collection	+ Add document	+ Start collection
chats >	cIkUXuR22pZtusVM0YUTii... >	messages
rooms		
users		
		+ Add field
		lastMessage: "how are you"
		timestamp: April 26, 2026 at 4:59:05 PM UTC+5
		▼ users
		0 "hOr64EyujeZJNN067cNJ6QD5TW13"
		1 "clkUXuR22pZtusVMOYUTiir3WsM2"

▼ app
> src
📄 build.gradle.kts
{ } google-services.json
> gradle
📄 build.gradle.kts
📄 gradle.properties
📄 local.properties
📄 my_app_android.iml
📄 settings.gradle.kts

dependencies:

```
flutter:
  sdk: flutter
firebase_core: ^2.30.0
firebase_auth: ^4.17.0
cloud_firestore: ^4.15.0
shared_preferences: ^2.2.2
particles_flutter: ^0.1.4
simple_animations: ^5.0.0
image_picker: ^1.1.2
firebase_storage: ^11.7.7
firebase_messaging: ^14.9.4
http: ^0.13.5
```

